

RepairHub

Projektdokumentation

Praxisarbeit der [Schule]

Dragan Gojic

Adresse: [ergänzen]

E-Mail-Adresse: [ergänzen]

Telefonnummer: [ergänzen]

Klasse: [ergänzen]

Abgabedatum: [ergänzen]

Qualifikationsreglement: Version [ergänzen]

Leitfaden für schriftliche Arbeiten: Version 2.1

1 Management Summary

[Vom Verfasser zu ergänzen: höchstens eine Seite zu Vorhaben, Zielen, Ergebnissen, Plattform, Mehrwert, grösstem Risiko und weiterem Vorgehen. Der Leitfaden verlangt hier persönliche Eigenleistung; dieser Abschnitt wurde nicht durch KI ausformuliert.]

Inhaltsverzeichnis

1 Management Summary.....	1
2 Projektgrundlagen und Planung.....	2
2.1 Ausgangslage und Projektauftrag.....	2
2.2 Zielsetzung und Abgrenzung.....	2
2.2.1 Muss-Ziele.....	3
2.2.2 Kann-Ziele.....	3
2.3 Vorgehensmodell und Zeitplanung.....	4
2.4 Projektrisiken.....	5
3 Anforderungsanalyse.....	6
3.1 Funktionale Anforderungen.....	6
3.2 Nichtfunktionale Anforderungen.....	9
3.3 Anwendungsfälle und Abnahmekriterien.....	10
4 Architektur und Lösungsentwurf.....	14
4.1 Systemarchitektur und Technologiewahl.....	14
4.2 Datenmodell.....	17
4.3 Ablaufdiagramme und Benutzeroberfläche.....	18
4.4 Bereitstellungsarchitektur und Sicherheit.....	20
5 Umsetzung und Bereitstellung.....	25
5.1 Projektstruktur und zentrale Implementierung.....	25
5.2 Geschäftslogik und REST API.....	25
5.3 Bereitstellung, Prüfzugänge und Repository.....	25
5.4 Betrieb während der Korrekturphase.....	25
6 Tests.....	26
6.1 Testvorgehen.....	26
6.2 Testprotokoll mit 6–12 Testfällen.....	26
6.3 Ergebnisse und offene Einschränkungen.....	26
7 Bedienungsanleitung.....	27
7.1 Bedienung der Webanwendung.....	27
7.2 API Authentifizierung und Endpunkte.....	27
8 Projektabschluss und Reflexion.....	27
8.1 Erreichung der Muss- und Kann-Ziele.....	27
8.2 Projektverlauf und kritische Würdigung der Lösung.....	27
8.3 Persönliche Reflexion und Ausblick.....	28
9 Abkürzungsverzeichnis.....	28
10 Glossar.....	28
11 Abbildungsverzeichnis.....	28
12 Tabellenverzeichnis.....	28
13 Quellenverzeichnis.....	29
Anhänge.....	29
Eigenständigkeitserklärung und Deklaration des KI-Einsatzes.....	30

2 Projektgrundlagen und Planung

Die Entwicklung von RepairHub umfasst die Planung, Umsetzung und Bereitstellung einer Webanwendung zur Verwaltung von Reparaturfällen für Haushaltsgeräte und Elektronik. Dieses Kapitel beschreibt die Ausgangslage, den Projektauftrag und die angestrebten Ziele. Zudem werden der Projektumfang, das geplante Vorgehen und die wesentlichen Projektrisiken festgelegt.

2.1 Ausgangslage und Projektauftrag

Bei der Bearbeitung von Reparaturfällen entstehen Informationen zu Geräten, Fehlerbildern, Reparaturschritten und benötigten Ersatzteilen. Werden diese Informationen getrennt festgehalten, erschwert dies die Übersicht über den Bearbeitungsstand und die voraussichtlichen Kosten.

Mit RepairHub wird eine Webanwendung entwickelt, welche diese Informationen zentral zusammenführt. Benutzer können Geräte und Reparaturfälle erfassen, Reparaturschritte planen, Ersatzteile zuordnen und den Reparaturstatus verfolgen. Die geschätzten Reparaturkosten werden aus Arbeitszeit und Ersatzteilen berechnet. Ausgewählte Reparaturdaten stehen zusätzlich über eine authentifizierte REST-API zur Verfügung.

Der Projektauftrag umfasst die Planung, Entwicklung, Prüfung und Bereitstellung der Anwendung. Im Mittelpunkt stehen eine verständliche Bedienung, eine konsistente Datenhaltung und ein nachvollziehbarer Aufbau der Software.

2.2 Zielsetzung und Abgrenzung

Ziel des Projekts ist die Entwicklung einer übersichtlichen Webanwendung, mit der Reparaturfälle von der Erfassung bis zum Abschluss dokumentiert werden können. Die Erfüllung der Projektziele wird anhand definierter Testfälle überprüft.

Die Projektziele werden in Muss- und Kann-Ziele unterteilt. Muss-Ziele bilden den verbindlichen Leistungsumfang. Kann-Ziele ergänzen die Anwendung und werden erst umgesetzt, wenn die Erfüllung der Muss-Ziele innerhalb der verfügbaren Zeit sichergestellt ist.

Der Projektumfang beschränkt sich auf die Verwaltung und Nachverfolgung von Reparaturfällen sowie die Abschätzung der Reparaturkosten. Eine vollständige Warenwirtschaft, die automatische Bestellung von Ersatzteilen, eine Zahlungsabwicklung und die Erstellung rechtsverbindlicher Rechnungen sind nicht Bestandteil des Projekts. Ebenso führt die Anwendung keine automatische Fehlerdiagnose durch. Fehlerbilder und Reparaturschritte werden durch die Benutzer erfasst.

2.2.1 Muss-Ziele

Die Muss-Ziele legen den verbindlichen Funktionsumfang von RepairHub fest. Ihre Erfüllung wird anhand definierter Testfälle überprüft.

ID	Muss-Ziel
M01	Benutzer können sich mit eindeutigem Benutzernamen und E-Mail-Adresse registrieren sowie passwortgeschützt anmelden.
M02	Geräte und Reparaturfälle können erfasst, angezeigt und bearbeitet werden. Die Daten werden dauerhaft gespeichert.
M03	Fehlerbilder und Reparaturschritte können dokumentiert und der Reparaturstatus kann nachgeführt werden.
M04	Ersatzteile können mit Preis und Menge einem Reparaturfall zugeordnet werden.
M05	Geschätzte Reparaturkosten werden aus Arbeitszeit, Stundensatz und Ersatzteilkosten berechnet.
M06	Ausgewählte Reparaturdaten sind über eine authentifizierte REST-API ohne Browser lesbar.
M07	Die Anwendung ist über das Internet erreichbar und über einen Webbrowser bedienbar.

Tabelle 1 Muss-Ziele von RepairHub

2.2.2 Kann-Ziele

Die Kann-Ziele verbessern die Bedienbarkeit und erweitern den Nutzen. K01 bis K04 wurden nach dem Muss-Umfang umgesetzt und in T12 gemeinsam abgenommen. Zusätzlich ist eine eigenständige Gerätesuche vorhanden. Die ursprüngliche Priorisierung bleibt als Planungsentscheidung nachvollziehbar.

ID	Kann-Ziel
K01	Reparaturfälle können nach Suchbegriffen und Bearbeitungsstatus gefiltert werden.
K02	Eine Übersichtsseite zeigt die Anzahl offener, laufender und abgeschlossener Reparaturfälle.
K03	Bilder können zu Geräten oder Fehlerbildern hochgeladen und angezeigt werden.
K04	Ein Reparaturbericht mit den wichtigsten Falldaten und geschätzten Kosten kann als PDF exportiert werden.

Tabelle 2 Kann-Ziele von RepairHub

2.3 Vorgehensmodell und Zeitplanung

Für RepairHub ist ein phasenorientiertes Vorgehen mit iterativer Umsetzung vorgesehen. Die Projektphasen schaffen eine nachvollziehbare Struktur von der Planung bis zum Abschluss. Innerhalb der Umsetzung werden einzelne Funktionen schrittweise entwickelt und getestet. Erkenntnisse aus den Tests können dadurch unmittelbar in die weitere Entwicklung einfließen.

Zunächst wurden Projektumfang und Anforderungen festgelegt. Das frühe Datenmodell diente dem Abgleich. Die tatsächliche Umsetzung folgte fachlichen Beispielen: Domain-Test, vertikaler Anwendungsfall mit Ports, konkrete Adapter und minimale Migration, anschliessend HTTP- und Browserprüfung. Dadurch entstand das Schema schrittweise aus den umgesetzten Anwendungsfällen.

Die Dokumentation wird während des gesamten Projekts laufend ergänzt. Auch die Bereitstellung wird frühzeitig erprobt, damit Probleme mit der Hostingumgebung rechtzeitig erkannt werden. Nach der Umsetzung der Muss-Ziele erfolgen die abschliessenden Tests und die Prüfung der Dokumentation.

Die folgende Aufteilung dient als Planung des verfügbaren Gesamtaufwands. Die konkreten Termine werden anhand des verbindlichen Projektendtermins festgelegt.

Phase	Wesentliche Arbeiten	Anteil	Meilenstein
Grundlagen und Analyse	Umfang, Ziele, Anforderungen und Risiken festlegen	15 %	Anforderungen festgelegt
Architektur und Entwurf	Datenmodell, Abläufe, Oberfläche und Bereitstellung planen	20 %	Lösungskonzept erstellt
Umsetzung	Muss-Funktionen schrittweise entwickeln und begleitend testen	35 %	Muss-Funktionen implementiert
Abschlussprüfung und Bereitstellung	Tests protokollieren, Fehler beheben, Internetzugriff und Dokumentation prüfen	20 %	Anwendung und Dokumentation fertiggestellt
Reserve	Unvorhergesehene Probleme bearbeiten; bei verbleibender Kapazität Kann-Ziele umsetzen	10 %	Projekt abgeschlossen

Tabelle 3 Aufwandsplanung

Die Fortschrittskontrolle erfolgt anhand der Meilensteine und des Umsetzungsstands der Muss-Ziele. Bei Verzögerungen werden zunächst die Kann-Ziele zurückgestellt. Die für Tests, Dokumentation und Bereitstellung vorgesehene Zeit wird bei der weiteren Planung berücksichtigt.

2.4 Projektrisiken

Die wesentlichen Projektrisiken betreffen den zeitlichen Aufwand, die technische Umsetzung und den zuverlässigen Betrieb der Anwendung. Die nachfolgende Einschätzung dient dazu, mögliche Schwierigkeiten frühzeitig zu berücksichtigen und geeignete Gegenmassnahmen einzuplanen. Sie wird im Projektverlauf überprüft und bei Bedarf angepasst.

Risiko	Eintritt	Auswirkung	Geplante Gegenmassnahme
Der Umsetzungsaufwand wird unterschätzt.	Mittel	Muss-Funktionen oder Dokumentation werden nicht rechtzeitig fertig.	Muss-Ziele priorisieren, Fortschritt anhand der Meilensteine prüfen und Zeitreserve einplanen.
Die Bereitstellung funktioniert in der Hostingumgebung nicht wie erwartet.	Mittel	Die Anwendung ist zum geplanten Termin nicht erreichbar.	Frühzeitig eine einfache Version mit Datenbankbindung und Unicorn bereitstellen und extern testen.
Fehler in Authentifizierung oder Zugriffsprüfung ermöglichen unberechtigte Datenzugriffe.	Mittel	Reparaturdaten sind unzureichend geschützt.	Passwörter als sichere Hashwerte speichern, etablierte Bibliotheken einsetzen und unerlaubte Zugriffe gezielt testen.
Fehlerhafte Eingaben oder Berechnungen führen zu falschen Reparaturkosten.	Mittel	Die angezeigten Ergebnisse sind unzuverlässig.	Eingaben validieren und Kostenberechnungen mit bekannten Ergebnissen und Grenzfällen prüfen.
Quellcode oder gespeicherte Daten gehen verloren.	Niedrig	Entwicklungsarbeit muss wiederholt werden oder Testdaten fehlen.	Änderungen regelmässig im Repository sichern und Datenbanksicherungen vor wesentlichen Änderungen erstellen.
Die Anwendung fällt während des Betriebs aus.	Mittel	Die Anwendung ist über Browser oder API zeitweise nicht erreichbar.	Erreichbarkeit regelmässig prüfen und das Vorgehen zum Neustart dokumentieren.

Tabelle 4 Projektrisiken und Gegenmassnahmen

3 Anforderungsanalyse

Die Anforderungsanalyse konkretisiert die Muss- und Kann-Ziele von RepairHub. Sie unterscheidet zwischen funktionalen Anforderungen, welche die Funktionen der Anwendung beschreiben, und nichtfunktionalen Anforderungen, welche deren Qualität festlegen. Jede Anforderung wird den zugehörigen Zielen zugeordnet. Positive und negative Szenarien beschreiben das erwartete Verhalten bei erfolgreicher Nutzung sowie bei Fehlern oder unzulässigen Aktionen.

3.1 Funktionale Anforderungen

Die funktionalen Anforderungen werden nach den Zielen M01 bis M07 und K01 bis K04 gruppiert. Die Anforderungen aus den Muss-Zielen bilden den verbindlichen Funktionsumfang. Anforderungen aus den Kann-Zielen werden bei ausreichendem Zeitbudget umgesetzt. Die eindeutigen Anforderungs-IDs ermöglichen eine spätere Zuordnung zu den Testfällen. In der ersten Tabellenspalte stehen die Anforderungs-ID und darunter der Zielbezug.

ID / Ziel	Funktionale Anforderung	Positives Szenario	Negatives Szenario
F01 M01	Registrierung mit eindeutigem Benutzernamen, eindeutiger E-Mail und Passwort (8–128 Zeichen); E-Mail-Bestätigung vor Anmeldung.	Vollständige und gültige Angaben führen zur Erstellung eines Kontos.	Fehlende Pflichtangaben, eine ungültige E-Mail-Adresse oder ein bereits verwendeter Benutzername verhindern die Registrierung.
F02 M01	Anmeldung mit E-Mail oder Benutzername und Passwort; Abmeldung beendet die Sitzung. Passwort-Recovery per E-Mail ist zusätzlich enthalten.	Korrekte Zugangsdaten ermöglichen die Anmeldung; die Abmeldung beendet die Sitzung.	Falsche Zugangsdaten werden abgewiesen. Nach der Abmeldung ist kein Zugriff auf geschützte Seiten möglich.
F03 M02	Geräte können mit Bezeichnung, Hersteller und Modell erfasst, angezeigt und bearbeitet werden.	Ein Gerät wird gespeichert; Änderungen werden nach erneutem Laden angezeigt.	Fehlende Pflichtangaben verhindern das Speichern.
F04 M02	Reparaturfälle können zu einem vorhandenen eigenen Gerät angelegt, angezeigt und bearbeitet werden.	Ein Reparaturfall wird dem ausgewählten Gerät zugeordnet.	Die Zuordnung zu einem nicht vorhandenen oder fremden Gerät wird abgewiesen.
F05 M03	Zu jedem Reparaturfall kann eine Fehlerbeschreibung erfasst und bearbeitet werden.	Eine Fehlerbeschreibung wird gespeichert und angezeigt.	Eine leere Fehlerbeschreibung wird zurückgewiesen.
F06 M03	Reparaturschritte können einem Reparaturfall zugeordnet, beschrieben und als erledigt gekennzeichnet werden.	Ein Schritt wird gespeichert und sein Erledigungszustand aktualisiert.	Ein Schritt ohne Beschreibung oder gültige Fallzuordnung wird nicht gespeichert.
F07 M03	Der Reparaturstatus kann auf «Offen», «In Bearbeitung» oder «Abgeschlossen» gesetzt werden. Neue Fälle erhalten den Status «Offen».	Der aktuelle Status erscheint übereinstimmend in Übersicht und Detailansicht.	Ein nicht zugelassener Statuswert wird abgewiesen.
F08 M04	Ersatzteile können mit Bezeichnung, Einzelpreis und Menge einem Reparaturfall zugeordnet und bearbeitet werden.	Ein Ersatzteil wird beim zugehörigen Reparaturfall angezeigt.	Eine fehlende Bezeichnung oder ungültige Fallzuordnung verhindert das Speichern.

ID / Ziel	Funktionale Anforderung	Positives Szenario	Negatives Szenario
F09 M04	Einzelpreise müssen nichtnegative Zahlen und Mengen positive ganze Zahlen sein.	Ein Einzelpreis von CHF 15 und eine Menge von 3 werden akzeptiert.	Negative Preise, Textwerte sowie Mengen von 0 oder 1,5 werden zurückgewiesen.
F10 M05	Arbeitszeit in Stunden und Stundensatz können je Reparaturfall als nichtnegative Zahlen erfasst werden.	Eine Arbeitszeit von 2 Stunden und ein Stundensatz von CHF 80 werden gespeichert.	Negative oder nichtnumerische Werte werden zurückgewiesen.
F11 M05	Die Gesamtkosten werden aus $\text{Arbeitszeit} \times \text{Stundensatz}$ und der Summe aus $\text{Ersatzteilmenge} \times \text{Einzelpreis}$ berechnet. Änderungen lösen eine Neuberechnung aus.	2 Stunden zu CHF 80 und 3 Ersatzteile zu CHF 15 ergeben CHF 205. Ohne Ersatzteile betragen die Kosten CHF 160.	Ein vom Browser übermittelter, manipulierter Gesamtbetrag wird nicht übernommen; die Berechnung erfolgt serverseitig.
F12 M06	Ein API-Client authentifiziert sich ohne Browser über einen zuvor generierten API-Key im Authorization-Bearer-Header.	Gültige Zugangsdaten ermöglichen authentifizierte Folgeanfragen.	Falsche Zugangsdaten werden abgewiesen.
F13 M06	Authentifizierte Clients können ausgewählte eigene Reparaturdaten mittels GET im JSON-Format abrufen.	Eine gültige Anfrage liefert die angeforderten eigenen Daten.	Fehlende oder ungültige Authentifizierung sowie Zugriffe auf fremde oder unbekannte Datensätze werden ohne Datenpreisgabe abgewiesen.
F14 M06	Die Reparaturendpunkte erlauben ausschliesslich lesenden Zugriff.	Unterstützte GET-Anfragen werden verarbeitet.	Schreibende Anfragen an diese Endpunkte werden abgewiesen und verändern keine Daten.
F15 M07	Die zentralen Funktionen sind über eine einheitliche Navigation im Webbrowser erreichbar.	Ein angemeldeter Benutzer kann Geräte und Reparaturfälle über die Oberfläche öffnen und bearbeiten.	Eine unbekannte URL führt zu einer verständlichen Fehlerseite mit Rückweg zur Anwendung.
F16 K01	Eigene Reparaturfälle können nach Suchbegriff und Bearbeitungsstatus gefiltert werden.	Passende Reparaturfälle werden angezeigt.	Ohne Treffer erscheint eine verständliche Meldung; fremde Fälle werden nicht angezeigt.
F17 K02	Eine Übersicht zeigt die Anzahl eigener Reparaturfälle je Bearbeitungsstatus.	Nach einem Statuswechsel werden die Kennzahlen beim nächsten Aufruf aktualisiert.	Ohne vorhandene Fälle werden Nullwerte angezeigt; fremde Fälle werden nicht mitgezählt.
F18 K03	Bilder können einem eigenen Gerät oder Reparaturfall zugeordnet werden. Zulässige Dateitypen und eine maximale Dateigrösse werden im Entwurf festgelegt.	Eine zulässige Bilddatei wird hochgeladen und angezeigt.	Unzulässige, beschädigte oder zu grosse Dateien werden mit einer Fehlermeldung zurückgewiesen.

ID / Ziel	Funktionale Anforderung	Positives Szenario	Negatives Szenario
F19 K04	Ein Reparaturbericht mit Falldaten und aktueller Kostenschätzung kann als PDF exportiert werden.	Der Bericht enthält die gespeicherten Daten des ausgewählten eigenen Reparaturfalls.	Der Export fremder oder nicht vorhandener Fälle wird verhindert.

Tabelle 5 Funktionale Anforderungen und Szenarien
Eigene Darstellung

3.2 Nichtfunktionale Anforderungen

Die nichtfunktionalen Anforderungen legen fest, wie zuverlässig, sicher und verständlich die Funktionen bereitgestellt werden. Sie sind ebenfalls aus den Projektzielen abgeleitet. Übergreifende Qualitätsanforderungen erhalten mehrere Zielbezüge und werden nur einmal aufgeführt. Ihre Gültigkeit für Kann-Funktionen setzt deren Umsetzung voraus.

ID / Ziel	Nichtfunktionale Anforderung	Positives Szenario	Negatives Szenario
N01 M01	Passwortsicherheit: Passwörter werden ausschliesslich als gesalzene Hashwerte mit einem etablierten Passwort-Hashverfahren gespeichert.	Die Anmeldung prüft das eingegebene Passwort gegen den gespeicherten Hash.	Die Datenbank enthält keine Klartextpasswörter; gleiche Passwörter erzeugen durch unterschiedliche Salts keine identischen gespeicherten Werte.
N02 M02– M04, M06; K01–K04	Persönliche Browser- und API-Zugänge sind eigentumsgebunden. Ausschliesslich der gesonderte System-Leseschlüssel darf Liste und Details aller Reparaturen lesen; keine Schreibrechte.	Eigene Daten können über die vorgesehenen Funktionen verwendet werden.	Manipulierte URLs, Formularwerte oder API-Anfragen ermöglichen weder das Lesen noch das Verändern fremder Daten.
N03 M02–M04	Datenintegrität: Zusammengehörige Änderungen werden vollständig oder gar nicht gespeichert. Beziehungen zwischen Datensätzen bleiben gültig.	Ein Reparaturschritt oder eine Ersatzteilzuordnung wird vollständig und korrekt gespeichert.	Ein provozierter Speicherfehler hinterlässt keine teilweise gespeicherten Änderungen oder ungültigen Zuordnungen.
N04 M02–M04	Dauerhafte Speicherung: Erfolgreich gespeicherte Daten bleiben nach einem Neustart der Anwendung erhalten.	Geräte, Reparaturfälle und zugehörige Angaben sind nach dem Neustart unverändert vorhanden.	Ein Neustart darf zuvor bestätigte Änderungen nicht verlieren lassen.
N05 M04, M05; K04	Berechnungsgenauigkeit: Geldbeträge werden mit geeigneter Dezimalarithmetik verarbeitet und nach einer einheitlichen Regel auf zwei Nachkommastellen ausgegeben.	Drei Ersatzteile zu CHF 0.10 ergeben CHF 0.30; Oberfläche und PDF zeigen denselben Betrag.	Rundungsartefakte dürfen weder falsche Summen noch unterschiedliche Beträge in den Ausgaben verursachen.
N06 M01– M05, M07; K01–K04	Bedienbarkeit: Beschriftungen und Rückmeldungen sind einheitlich und verständlich. Bei Eingabefehlern bleiben gültige Angaben soweit möglich erhalten; Passwörter werden nicht erneut ausgegeben.	Nach einer Korrektur kann ein Formular ohne vollständige Neueingabe gespeichert werden.	Eine ungültige Eingabe führt weder zu einer technischen Fehlerseite noch zu einer irreführenden Erfolgsmeldung.
N07 M01, M06, M07	Übertragungssicherheit: Weboberfläche und API werden über HTTPS bereitgestellt.	Browser und API-Client können eine gültig abgesicherte Verbindung herstellen.	Über einen unverschlüsselten HTTP-Zugang darf keine Anmeldung verarbeitet werden.

ID / Ziel	Nichtfunktionale Anforderung	Positives Szenario	Negatives Szenario
N08 M07	Verfügbarkeit und Wiederherstellung: Die Anwendung bleibt mindestens vier Wochen nach Abgabe erreichbar. Ein Neustart ist anhand dokumentierter Schritte möglich.	Die Anwendung ist von einem externen Anschluss erreichbar und kann nach einem kontrollierten Stopp wieder gestartet werden.	Ein erkannter Ausfall wird untersucht und behoben; eine Wiederherstellung darf keine undokumentierten Einrichtungsschritte erfordern.
N09 M01–M07	Fehlerbehandlung: Technische Fehler werden nachvollziehbar protokolliert. Benutzer erhalten verständliche Meldungen ohne interne Details.	Ein Fehler kann anhand der Protokollierung untersucht werden.	Bei einem Datenbankausfall werden weder Passwörter noch Zugangsschlüssel oder interne Fehlerberichte an Benutzer ausgegeben.
N10 M03– M06; K04	Wartbarkeit: Fachlogik, Datenzugriff und Darstellung sind nachvollziehbar gegliedert. Die Kostenberechnung wird zentral umgesetzt.	Oberfläche, API und gegebenenfalls PDF verwenden dieselbe Berechnungslogik.	Eine Codeprüfung darf keine voneinander abweichenden Kopien der Kostenberechnung feststellen.

Tabelle 6 Nichtfunktionale Anforderungen und Szenarien

Eigene Darstellung

3.3 Anwendungsfälle und Abnahmekriterien

Die Anwendungsfälle verbinden die funktionalen und nichtfunktionalen Anforderungen zu überprüfbaren Abläufen. Die beiden Use-Case-Diagramme zeigen die beteiligten Rollen, die Systemgrenze und die fachliche Gruppierung. Die Anforderungs-IDs stellen den Bezug zu den Tabellen in den Abschnitten 3.1 und 3.2 her. Die Reparaturübersicht gehört zum Muss-Ziel M02; die gekennzeichneten Erweiterungen gehören zu den Kann-Zielen.

Die Übersicht grenzt Identität und Zugriff, Geräteverwaltung, Reparaturverwaltung, Ersatzteile und Kosten sowie die lesende REST-API voneinander ab. Diese fachlichen Bereiche dienen als Ausgangspunkt für die spätere Komponentenaufteilung. Geschützte Funktionen setzen eine gültige Anmeldung beziehungsweise API-Authentifizierung voraus und erlauben nur den Zugriff auf eigene Daten.

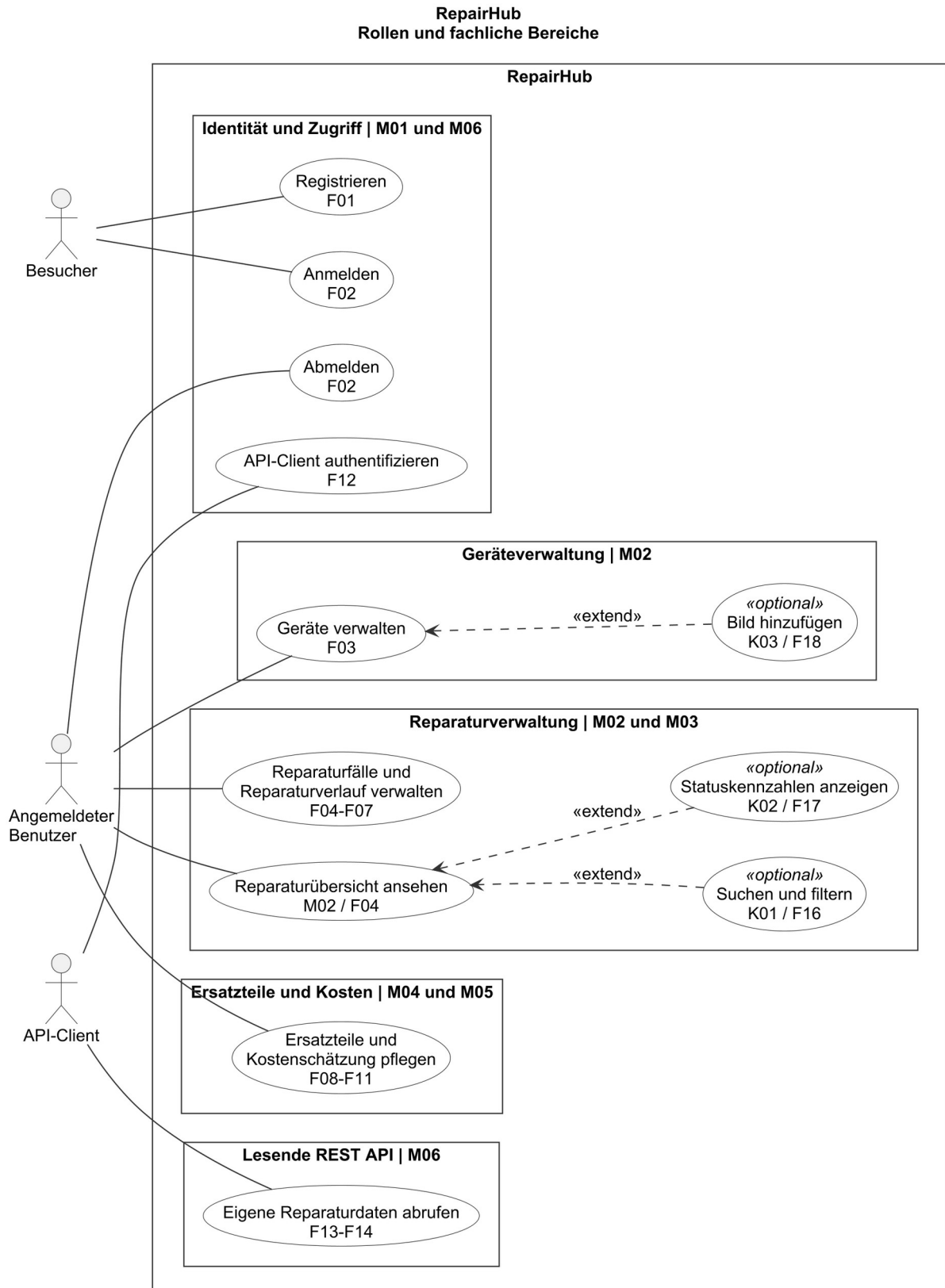


Abbildung 1 Use-Case-Übersicht zu Rollen und Bereichen
Eigene Darstellung

Das Detaildiagramm zeigt die Beziehungen bei der Reparaturbearbeitung. «include» kennzeichnet zwingendes Teilverhalten, beispielsweise die Zuordnung eines Geräts beim Anlegen eines Reparaturfalls. «extend» kennzeichnet eine bedingte Ergänzung, beispielsweise den angeforderten PDF-Export. Fehlerfälle und unzulässige Aktionen werden in den Anforderungstabellen beschrieben und bei der Abnahme berücksichtigt.

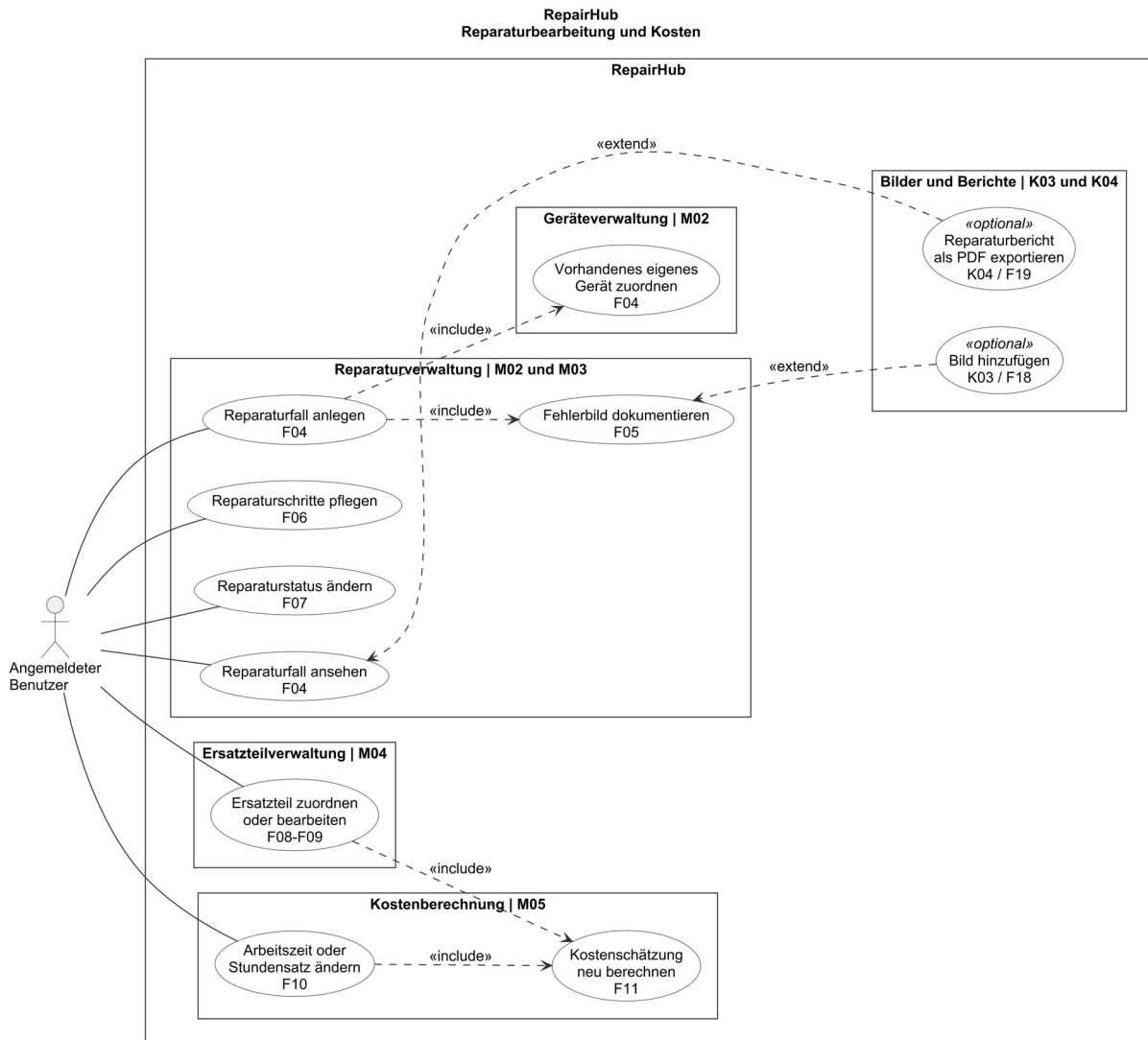


Abbildung 2 Use-Case-Details zu Reparaturbearbeitung und Kosten
Eigene Darstellung

Die folgenden Abnahmekriterien fassen die erwarteten Ergebnisse der zentralen Anwendungsfälle zusammen. Jeder Ablauf berücksichtigt sowohl die erfolgreiche Nutzung als auch relevante Fehlerfälle. Die referenzierten Anforderungs-IDs ermöglichen die Zuordnung der späteren Testergebnisse zu den Projektzielen.

Anwendungsfall	Anforderungen	Abnahmekriterium
Benutzerkonto erstellen und verwenden	F01–F02, N01, N06–N07	Eine gültige Registrierung und Anmeldung funktionieren. Fehlerhafte Angaben werden abgewiesen, Passwörter sicher gespeichert und die Sitzung bei der Abmeldung beendet.
Gerät und Reparaturfall verwalten	F03–F04, N02–N04, N06	Eigene Datensätze können erstellt und bearbeitet werden. Ungültige Zuordnungen und Fremdzugriffe werden verhindert. Bestätigte Änderungen bleiben nach einem Neustart erhalten.
Reparaturverlauf dokumentieren	F05–F07, N02–N04, N06	Fehlerbeschreibung, Reparaturschritte und Status werden korrekt gespeichert und angezeigt. Leere Pflichtangaben, ungültige Statuswerte und fremde Fallzuordnungen werden abgewiesen.
Ersatzteile zuordnen und Kosten berechnen	F08–F11, N02–N06, N10	Gültige Ersatzteile und Arbeitswerte ergeben den erwarteten Gesamtbetrag. Änderungen aktualisieren die Berechnung. Ungültige Werte und manipulierte Gesamtbeträge werden nicht übernommen.
Reparaturdaten über die API lesen	F12–F14, N02, N07, N09	Authentifizierung und Datenabruf funktionieren ohne Browser. Ungültige Zugriffe und schreibende Anfragen werden ohne unerlaubte Datenänderung oder Datenpreisgabe abgewiesen.
Anwendung über das Internet bedienen	F15, N06–N09	Die Anwendung ist extern über HTTPS bedienbar. Unbekannte URLs und technische Fehler werden verständlich behandelt. Ein kontrollierter Neustart ist erfolgreich.
Suche und Übersicht verwenden, sofern umgesetzt	F16–F17, N02, N06	Suchergebnisse und Kennzahlen stimmen mit den eigenen gespeicherten Fällen überein. Leere Ergebnisse werden verständlich dargestellt.
Bilder und PDF verwenden, sofern umgesetzt	F18–F19, N02, N05–N06	Zulässige Bilder werden korrekt zugeordnet und PDF-Berichte enthalten die aktuellen eigenen Falldaten. Unzulässige Dateien und fremde Zugriffe werden verhindert.

Tabelle 7 Anwendungsfälle und Abnahmekriterien

Eigene Darstellung

Ein Anwendungsfall gilt als erfüllt, wenn die zugeordneten Anforderungen und die ausgewählten positiven und negativen Szenarien das erwartete Ergebnis liefern. Im Testprotokoll werden Durchführung, Soll- und Ist-Ergebnis sowie festgestellte Abweichungen dokumentiert. Nicht erfüllte Anforderungen werden mit ihren Auswirkungen und ihrem Bearbeitungsstand ausgewiesen.

4 Architektur und Lösungsentwurf

Die Architektur von RepairHub legt fest, wie die Anforderungen technisch umgesetzt werden. Grundlage bilden die fachlichen Bereiche der Anforderungsanalyse: Benutzerverwaltung, Geräteverwaltung, Reparaturverwaltung, Ersatzteile und Kosten sowie die REST-API. Der Entwurf beschreibt deren Zusammenspiel, die Datenhaltung und die Bereitstellung der Anwendung.

4.1 Systemarchitektur und Technologiewahl

RepairHub ist ein modularer Flask-Monolith. Die Fachdomänen users, devices, repairs, parts und costs liegen unter app.domains und enthalten vertikale Anwendungsfälle. Domain-Handler kennen nur Standardbibliothek, eigene Ports und DTOs; Flask und Datenbank bleiben ausserhalb des Fachkerns. app.bootstrap erzeugt die Adapter und injiziert sie. Der Aufrufweg ist Präsentation → Handler → Port → Adapter (RepairHub, 2026a).

Präsentation, fachliche Anwendungsfälle und technische Adapter bleiben getrennte Verantwortlichkeiten. Die neuere Benutzerentscheidung dreht die Quellcode-Abhängigkeit um: Ein Datenadapter implementiert den Port des Anwendungsfalls. Fachdomänen importieren weder Flask noch SQLAlchemy noch konkrete Adapter. Zwischen Slices sind keine Imports zulässig. repairs und parts dürfen die reine Kostenberechnung verwenden (RepairHub, 2026a).

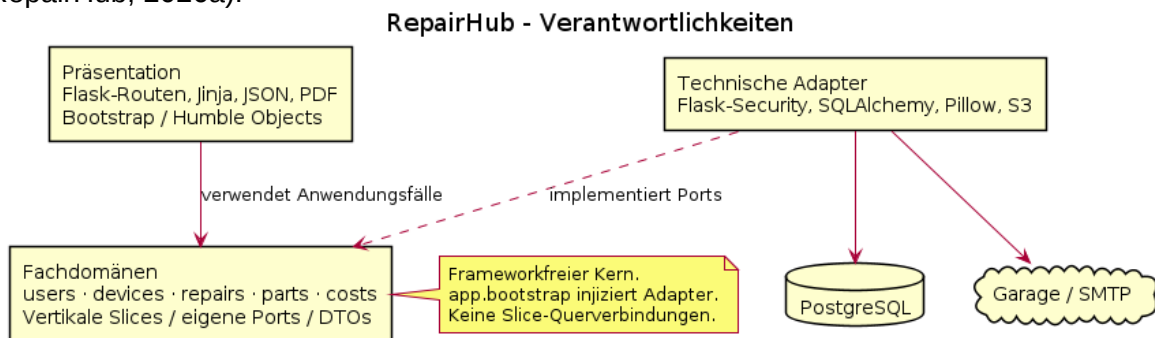


Abbildung 3 Verantwortlichkeiten und Abhängigkeitsumkehr

Eigene Darstellung

Die fachlichen Bereiche werden innerhalb dieser Schichten als Module umgesetzt. Weboberfläche und REST-API verwenden dieselbe Anwendungslogik. Insbesondere die Eigentumsprüfung und Kostenberechnung werden zentral implementiert. Die Kostenberechnung erhält validierte Arbeitswerte und Ersatzteilpositionen; ein eigener Datenbankzugriff ist dafür nicht erforderlich.

RepairHub - Quellcode-Abhängigkeiten

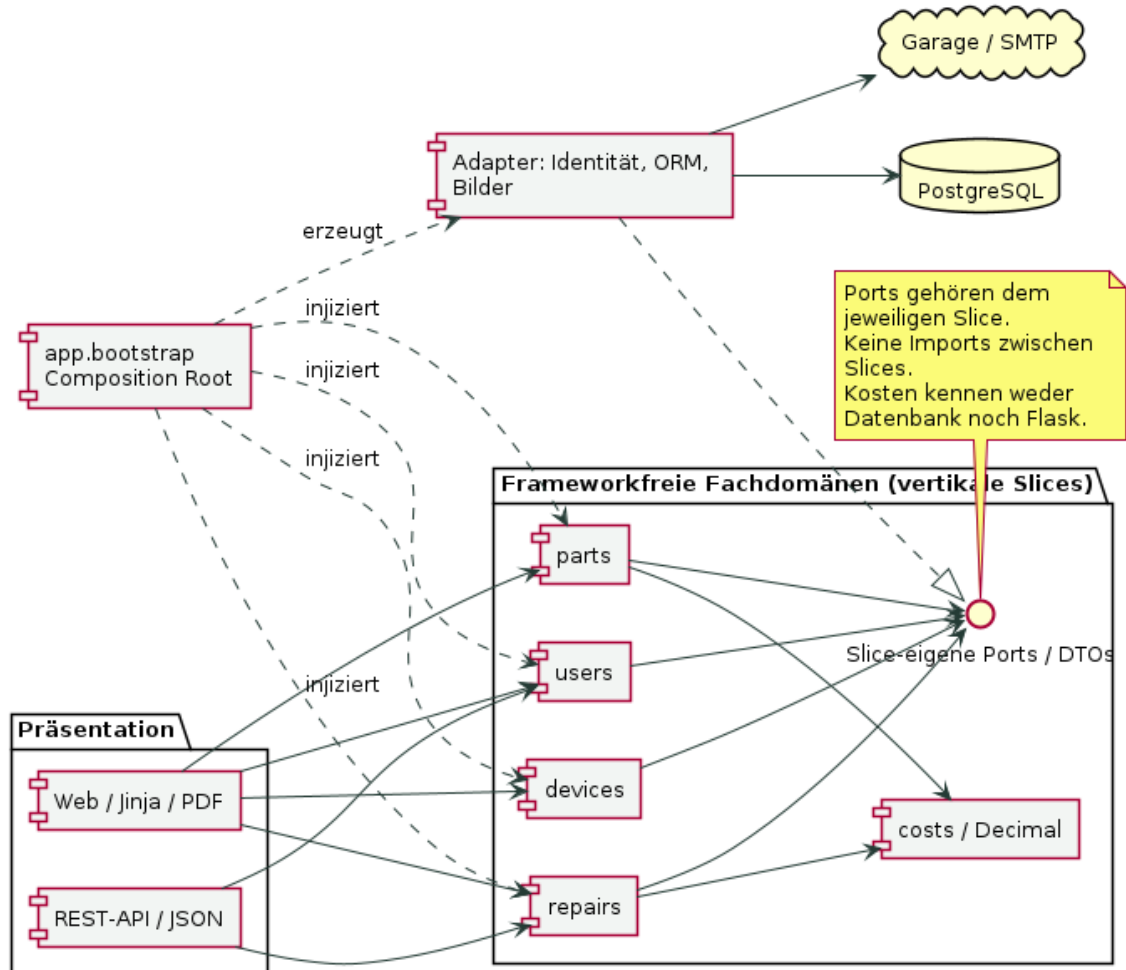


Abbildung 4 Komponenten und Abhängigkeiten von RepairHub

Eigene Darstellung

Der modulare Aufbau unterstützt die Wartbarkeit, da fachliche Änderungen einem klaren Bereich zugeordnet werden können. Gleichzeitig bleibt die Bereitstellung einfacher als bei mehreren unabhängig betriebenen Diensten. Eine Einschränkung besteht darin, dass die Module gemeinsam bereitgestellt werden und ein Ausfall der Anwendung sämtliche Funktionen betrifft.

Für eine spätere Skalierung können mehrere Anwendungsprozesse eingesetzt werden. Dabei müssen insbesondere Datenbankverbindungen, Sitzungen und gegebenenfalls hochgeladene Dateien berücksichtigt werden. Eine verteilte Bereitstellung ist für die erste Version nicht vorgesehen.

Die folgende Technologieauswahl ist umgesetzt. Python 3.13.15 und die Abhängigkeiten sind im Repository festgelegt. Bootstrap wird lokal ausgeliefert; kleine JavaScript-DOM-Adapter binden unabhängig testbare Presenter an. Flask-Security übernimmt die technischen Identitätsfunktionen hinter injizierten Benutzer-Ports. Pillow und Garage ergänzen Bildverarbeitung und privaten Speicher, ReportLab die PDF-Darstellung (RepairHub, 2026a).

Technologie	Einsatzzweck und Begründung
Python und Flask	Umsetzung der HTTP-Endpunkte und der Anwendung. Flask unterstützt die modulare Gliederung von Routen und die Einbindung von Vorlagen (Pallets, o. J.).
Jinja2, HTML und CSS	Serverseitige Darstellung der Weboberfläche ohne zusätzliches Frontend-Framework. Jinja ist in Flask integriert (Pallets, o. J.).
PostgreSQL	Relationale Speicherung der verknüpften Fachdaten. NUMERIC ermöglicht die genaue Verarbeitung von Geldbeträgen (PostgreSQL Global Development Group, o. J.).
SQLAlchemy	Abbildung der Datenbanktabellen auf Python-Modelle und strukturierter Datenzugriff (SQLAlchemy, o. J.).
Flask-Migrate	Nachvollziehbare Verwaltung von Änderungen am Datenbankschema mit Alembic (Grinberg, o. J.).
Flask-Security und Flask-Login	Bibliotheksintegration für Registrierung, E-Mail-Verifikation, Passwort-Recovery und Sitzungen; Aufruf über injizierte Benutzer-Ports (RepairHub, 2026a).
Flask-WTF	Verarbeitung von Webformularen und Schutz vor gefälschten Formularanfragen durch CSRF-Schutz (Flask-WTF, o. J.).
Gunicorn und Nginx	Gunicorn führt die Flask-Anwendung aus; Nginx übernimmt den öffentlichen Zugang und leitet Anfragen an die Anwendung weiter (Pallets, o. J.; F5, o. J.).
Docker Engine und Docker Compose	Vier technische Produktionsdienste: nginx, app, db und garage. Gemeinsame Compose-Grundlage; Ansible gleicht Release-Digests, Konfiguration und Migrationen idempotent ab. Nur Releases veröffentlichen und deployen (RepairHub, 2026a).

Tabelle 8 Technologieauswahl und Begründung
Eigene Darstellung

4.2 Datenmodell

Das nachfolgende persistente Modell entspricht den implementierten Anwendungsfällen. Neben Benutzer, Gerät, Reparatur, Schritt und Ersatzteilposition umfasst es Bildmetadaten sowie technische Rollenverknüpfungen für Flask-Security. Diese technischen Tabellen schaffen keine Administratoroberfläche. Metadaten liegen in PostgreSQL, die Bildobjekte in Garage (RepairHub, 2026a).

RepairHub - persistentes Modell (Auszug der Spalten)

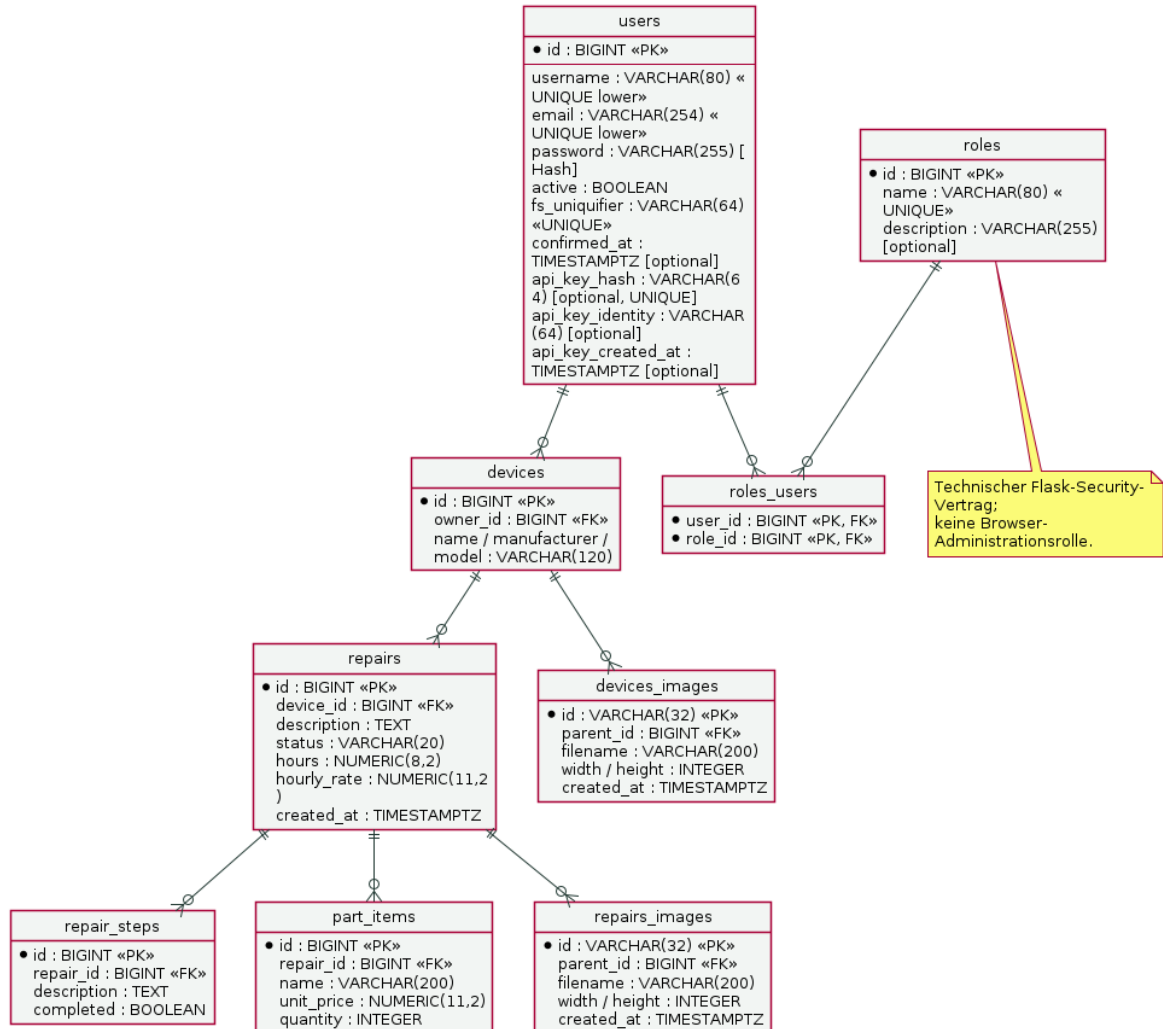


Abbildung 5 Relationales Datenmodell von RepairHub
Eigene Darstellung

Die Eigentümerschaft eines Reparaturfalls wird über das zugehörige Gerät bestimmt. Reparaturschritte und Ersatzteilpositionen übernehmen diese Zuordnung über ihren Reparaturfall. Eine zusätzliche Speicherung der Benutzer-ID in jeder Tabelle ist dadurch nicht erforderlich.

Ersatzteile werden zunächst als Positionen innerhalb eines Reparaturfalls gespeichert. Ein zentraler Ersatzteilkatalog mit Lagerbeständen ist nicht vorgesehen. Dadurch bleibt das Datenmodell auf die Reparaturverwaltung beschränkt.

Die geschätzten Gesamtkosten werden aus den gespeicherten Berechnungsgrundlagen ermittelt: $\text{Gesamtkosten} = \text{Arbeitszeit} \times \text{Stundensatz} + \text{Summe aus Ersatzteilmenge} \times \text{jeweiligem Einzelpreis}$. Der Gesamtbetrag wird nicht zusätzlich als unabhängig bearbeitbarer Wert gespeichert. Dadurch werden widersprüchliche Werte nach Änderungen an Arbeitszeit oder Ersatzteilen vermieden.

Geldbeträge und Arbeitszeit werden mit Decimal und PostgreSQL NUMERIC verarbeitet. Die Gesamtsumme wird aus den validierten Grundlagen abgeleitet und mit ROUND_HALF_UP auf zwei Nachkommastellen ausgegeben. Geldbeträge sind CHF; API-Dezimalwerte werden als Strings übertragen. Browser, API und PDF nutzen dieselbe serverseitige Berechnung (RepairHub, 2026a).

Fremdschlüssel, Pflichtfelder, Checks und eindeutige Indizes sichern persistente Regeln. Benutzername und E-Mail-Adresse sind unabhängig von Gross-/Kleinschreibung eindeutig. Arbeitszeit, Stundensatz und Einzelpreis sind nichtnegativ; Mengen sind positive ganze Zahlen. Die Zustände lauten open, in_progress und completed. Eigentum wird über Gerät und Fall abgeleitet. Der zentrale System-Leseschlüssel darf auf ausdrücklichen Benutzerauftrag alle Fälle lesen; persönliche Zugänge bleiben auf eigene Daten beschränkt.

4.3 Ablaufdiagramme und Benutzeroberfläche

Der typische Ablauf beginnt mit Registrierung und E-Mail-Bestätigung. Nach der Anmeldung öffnet der Benutzer seine Geräte oder Reparaturen über die Navigation. Ein neues Gerät benötigt Name, Hersteller und Modell; ein neuer Reparaturfall eine Fehlerbeschreibung. Die deutschsprachige Schritt-für-Schritt-Anleitung mit echten Playwright-Aufnahmen liegt unter docs/user-guide/README.md (RepairHub, 2026c).

Ansicht	Inhalt und mögliche Aktionen
Registrierung und Anmeldung	Benutzerkonto erstellen und Zugang zur Anwendung erhalten.
Geräteübersicht	Eigene Geräte anzeigen und neue Geräte erfassen.
Geräteansicht	Gerätedaten bearbeiten und zugehörige Reparaturfälle anzeigen.
Reparaturübersicht	Eigene Reparaturfälle mit Gerät und aktuellem Status anzeigen.
Reparaturdetailansicht	Fehlerbild, Reparaturschritte, Ersatzteile, Arbeitswerte und Status bearbeiten sowie Kosten anzeigen.

Tabelle 9 Ansichten der Benutzeroberfläche

Eigene Darstellung

Neue Reparaturfälle erhalten den Status «Offen». Für die weitere Bearbeitung stehen «In Bearbeitung» und «Abgeschlossen» zur Verfügung. Ein abgeschlossener Fall kann zur erneuten Bearbeitung wieder auf «In Bearbeitung» gesetzt werden. Weitere Einschränkungen der Statuswechsel sind zunächst nicht vorgesehen.

Das Aktivitätsdiagramm zeigt die Bearbeitung einschliesslich Fehlerpfaden. Anmeldung und Eigentümerschaft werden vor der Anzeige und erneut beim Speichern geprüft. Ungültige Eingaben führen zur Korrektur zurück. Gültige Änderungen werden innerhalb einer Transaktion gespeichert; bei einem Speicherfehler werden sie zurückgerollt. Kostenrelevante Änderungen lösen eine zentrale Neuberechnung aus.

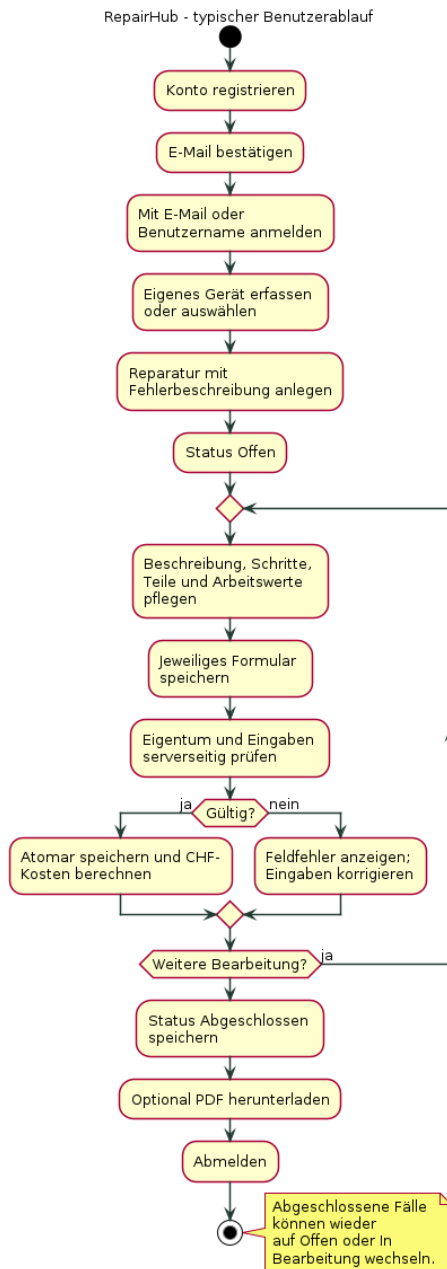


Abbildung 6 Aktivitätsdiagramm zur Reparaturbearbeitung
Eigene Darstellung

4.4 Bereitstellungsarchitektur und Sicherheit

RepairHub wird mit Docker Compose auf einem Debian-12-Host bereitgestellt. Vier getrennte technische Dienste übernehmen Nginx, Unicorn/Flask, PostgreSQL und Garage. App, Datenbank und S3 bleiben privat; Nginx veröffentlicht HTTPS und den HTTP-Zugang für Weiterleitung und ACME. GitHub Actions liefert freigegebene Releases über Ansible aus. Der Zielhost baut das App-Image nicht erneut (RepairHub, 2026a).

Nginx nimmt HTTPS-Anfragen über den veröffentlichten Port 443 entgegen. Innerhalb des Docker-Netzwerks erreicht Nginx die Anwendung über app:8000 und die Anwendung PostgreSQL über db:5432. Für Anwendung und Datenbank werden keine Host-Ports veröffentlicht. Die Dienstenamen ermöglichen die Kommunikation unabhängig von wechselnden Container-IP-Adressen (Docker Inc., o. J. a).

RepairHub - tatsächliche Bereitstellung

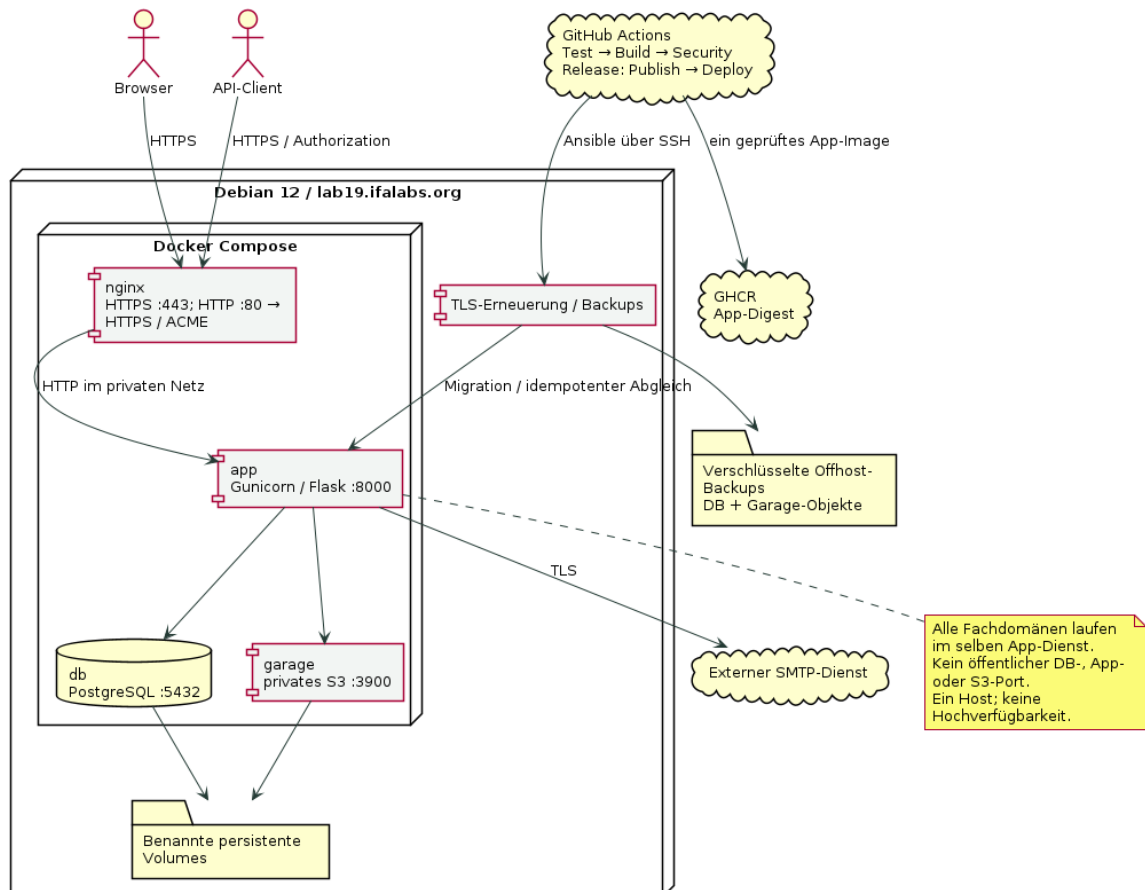


Abbildung 7 Bereitstellung von RepairHub mit Docker Compose
Eigene Darstellung

Entwicklung und Produktion verwenden dieselbe `compose.yaml` als Grundlage, dieselben festgelegten Softwareversionen und denselben Aufbau der Dienste. Ergänzende Dateien `compose.development.yaml` und `compose.production.yaml` enthalten nur die jeweiligen Abweichungen. Lokal kann der Quellcode für schnelle Änderungen eingebunden werden. Vor der Bereitstellung wird das gebaute Anwendungs-Image getestet und unverändert in der Produktion eingesetzt. Dort werden keine Quellcode-Verzeichnisse eingebunden; Debug-Modus und automatisches Neuladen sind deaktiviert (Docker Inc., o. J. b).

Umgebungsspezifische Werte wie Domain, TLS-Zertifikate und Zugangsdaten werden getrennt vom Anwendungs-Image bereitgestellt. Geheimnisse werden ausserhalb des Repositorys verwaltet. Die gemeinsame Grundlage hält die Entwicklungsumgebung nahe am produktiven Betrieb; produktionsspezifische Einstellungen werden zusätzlich vor der Freigabe geprüft.

Die PostgreSQL-Daten liegen in einem benannten Volume und bleiben beim Ersetzen eines Containers erhalten. Das Volume ersetzt keine Sicherung: Datenbanksicherungen werden ausserhalb des Hosting-Servers aufbewahrt und die Wiederherstellung wird geprüft. Bei einem Serverwechsel werden neben der Compose-Konfiguration und den Images auch Daten, Geheimnisse und Zertifikate übernommen (Docker Inc., o. J. c).

Docker Compose vereinfacht die wiederholbare Bereitstellung und den Wechsel auf einen geeigneten Server. Der einzelne Host bleibt jedoch ein zentraler Ausfallpunkt. Neustartrichtlinien, Erreichbarkeitskontrollen und geprüfte Sicherungen unterstützen den Betrieb; eine redundante Infrastruktur ist für die erste Version nicht vorgesehen.

Die Weboberfläche verwendet bestätigte Browsersitzungen und CSRF-geschützte Formulare. Ein Benutzer generiert einen persönlichen API-Schlüssel im Frontend. Der Klartext erscheint einmal; gespeichert werden Hash und Identitätsbindung. Der API-Client sendet Authorization: Bearer <api-key>. Es existiert kein POST /api/auth/token. Persönliche Schlüssel bleiben bis zu Widerruf oder Entzug der Identitätsgültigkeit nutzbar. API_SMOKE_KEY ist ein gesonderter systemweiter Leseschlüssel ohne Browser-Administratorrolle (RepairHub, 2026a).

Die erste API-Sequenz zeigt die einmalige Erzeugung des persönlichen Schlüssels. Danach erfolgen Datenabrufe ohne Browsercookies und ohne erneute Passwortübergabe. Ersetzen oder Widerrufen macht den alten Schlüssel unmittelbar ungültig.

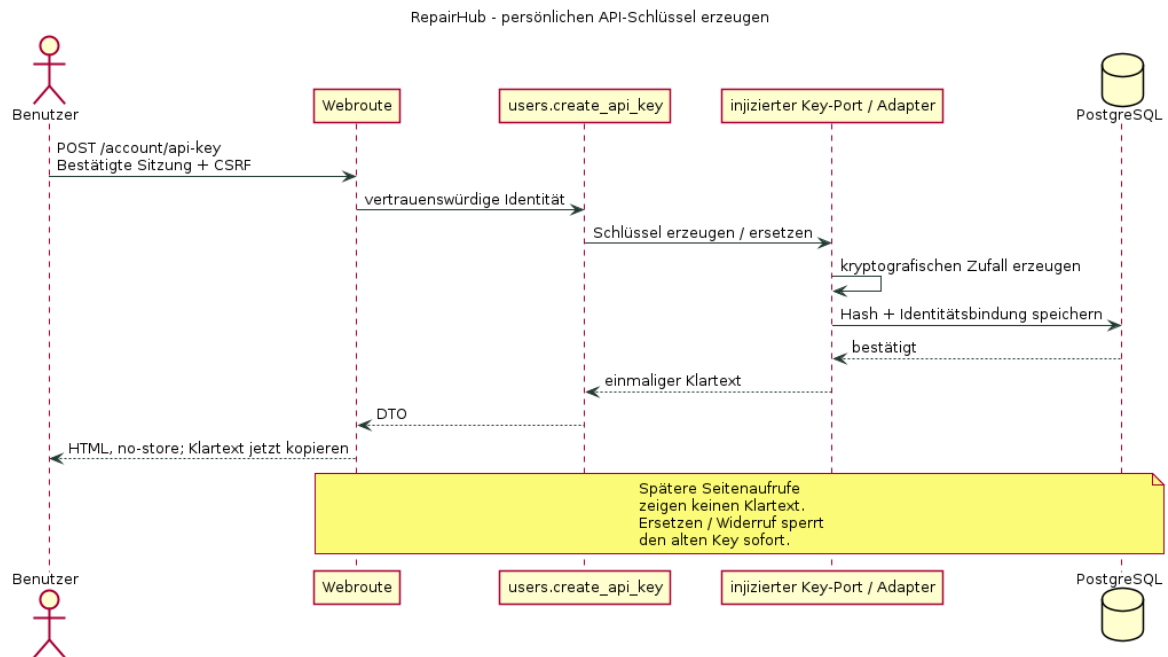


Abbildung 8 Persönlichen API-Schlüssel erzeugen
Eigene Darstellung

Die zweite Sequenz zeigt den Detailabruf. Der Key-Adapter ermittelt eine persönliche oder eine explizite System-Leseberechtigung. Persönliche Abfragen werden nach Eigentümer gefiltert; unbekannte und fremde Fälle liefern gleichartig 404. Die Fallauskunft berechnet die aktuellen Kosten über costs. Die API selbst greift weder auf ORM-Modelle noch direkt auf die Kostenkomponente zu.

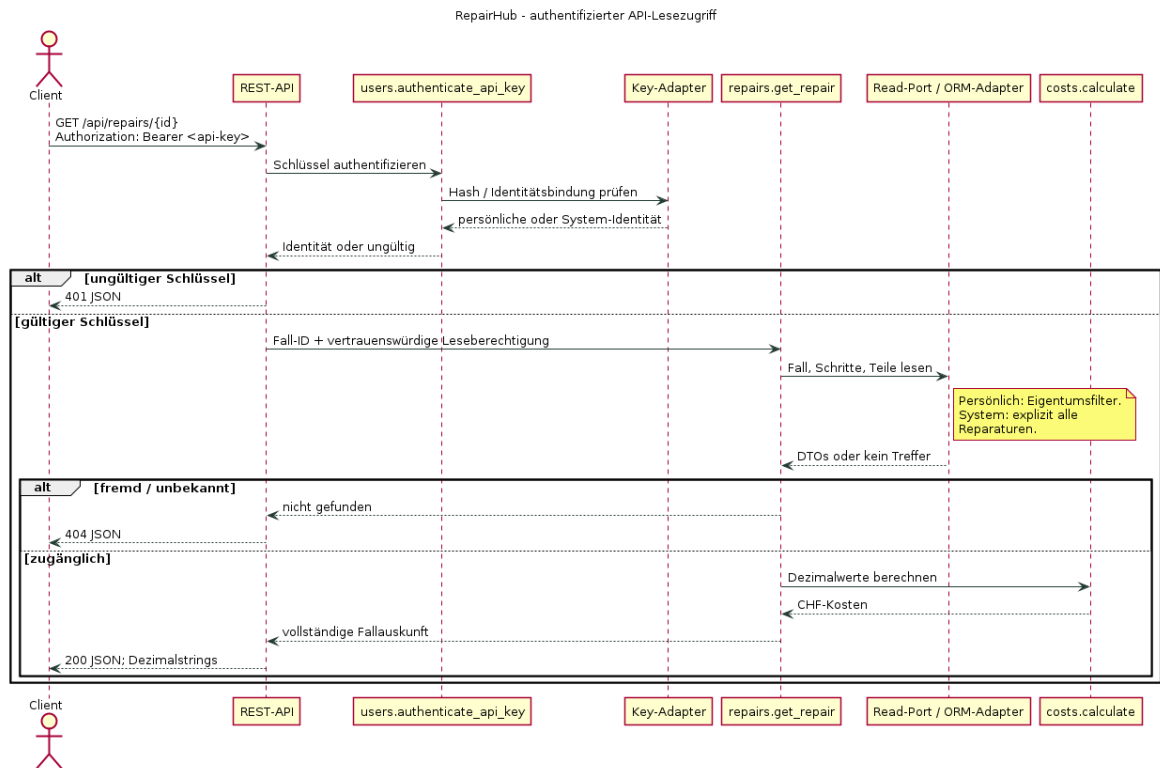


Abbildung 9 API-Sequenz zum geschützten Datenabruf
Eigene Darstellung

Die folgenden Massnahmen sind umgesetzt und in T12 geprüft. Die begrenzten PostgreSQL-gosu-Ausnahmen bleiben bis zum dokumentierten Ablauf ausdrücklich ausgewiesen. Eine tatsächliche Datenbankwiederherstellung wurde für die Praxisarbeit auf Benutzerentscheid nicht ausgeführt; das Verfahren ist dokumentiert. Die vierwöchige Betriebsfrist bleibt bis zu ihrem realen Ablauf offen (RepairHub, 2026b).

Sicherheitsbereich	Vorgesehene Massnahme
Passwörter	Speicherung als gesalzene Hashwerte mit einem etablierten Passwort-Hashverfahren.
Übertragung	HTTPS für Weboberfläche und API; keine Verarbeitung von Anmeldungen über unverschlüsseltes HTTP.
Webformulare	Serverseitige Validierung und CSRF-Schutz.
Sitzungen	Cookies mit Secure, HttpOnly und geeignetem SameSite-Attribut; Beendigung der Sitzung bei Abmeldung.
API-Zugriff	Persönliche API-Keys: Hash und Identitätsbindung; nur Authorization: Bearer. Separater System-Leseschlüssel mit globalem Zugriff. Keine schreibende Reparatur-API.
Konfiguration	Geheimnisse ausserhalb von Repository und Images bereitstellen; umgebungsspezifische Konfiguration getrennt verwalten.
Fehlerbehandlung	Debug-Modus im Betrieb deaktivieren und technische Fehler ohne Geheimnisse protokollieren.
Datenbank	Eingeschränktes Anwendungskonto; Zugriff über das Docker-Netzwerk ohne veröffentlichten Host-Port.
Wiederherstellung	Benannte Volumes; verschlüsselte zusammengehörige DB-/Garage-Backups ausserhalb des Hosts. Restore-Anleitung vorhanden, tatsächliche Wiederherstellung ausdrücklich nicht ausgeführt.

Tabelle 10 Sicherheitsmassnahmen für RepairHub

Eigene Darstellung

5 Umsetzung und Bereitstellung

Die technische Umsetzung ist anhand des Repository-Stands und der T12-Abnahme dokumentiert. Die folgenden Abschnitte führen Quellstruktur, Schnittstellen und die für die Prüfung nötigen Zugangsangaben zusammen.

5.1 Projektstruktur und zentrale Implementierung

app.domains enthält die fünf frameworkfreien Fachdomänen mit Slice-Handlern, Ports und DTOs. app.data implementiert Persistenz- und Dateiadapter; app.adapters.users kapselt Flask-Security und API-Keys. app.web und app.api übersetzen HTTP in Anwendungsfallaufrufe. app.bootstrap verdrahtet diese Komponenten. scripts/check_architecture.py und positive/negative Architekturtests sichern die Importgrenzen.

Das Frontend verwendet lokal versioniertes Bootstrap und Humble Objects: DOM-freie Presenter koordinieren Verhalten, kleine Views bedienen DOM und Fetch. Geräte- und Reparaturlisten starten mit 20 serverseitigen Zeilen und halten mit JavaScript höchstens 60 Zeilen. Suchanfragen reagieren nach 300 ms Eingabepause; Statusfilter sofort. Ohne JavaScript bleiben HTML-Formulare und Seitenlinks nutzbar.

5.2 Geschäftslogik und REST API

Eigentumsprüfung, Eingabegrenzen und Transaktionen werden in den jeweiligen Anwendungsfällen und Adaptionen gemeinsam für Web und API verwendet. Reparaturen und Teile rufen dieselbe reine Decimal-Kostenberechnung auf. 2 Stunden zu CHF 80 plus 3 Teile zu CHF 15 ergeben CHF 205.00. Der Gesamtbetrag wird nicht unabhängig gespeichert.

Die API bietet GET /api/repairs und GET /api/repairs/{id}. Jeder Aufruf benötigt Authorization: Bearer <api-key>. Liste und Detail sind lesend. Persönliche Schlüssel beschränken sich auf eigene Fälle; API_SMOKE_KEY ist der ausdrücklich vereinbarte systemweite Lesezugang. Eine Anmeldung über Benutzername und Passwort an einem Token-Endpunkt entfällt.

5.3 Bereitstellung, Prüfzugänge und Repository

Öffentlicher Zugang: <https://lab19.ifalabs.org>, HTTPS-Port 443. HTTP-Port 80 dient Weiterleitung und Zertifikatsvalidierung; App, PostgreSQL und Garage besitzen keine öffentlichen Ports. Repository: <https://github.com/GojicDragan/RepairHub>; am 22. September 2026 öffentlich lesbar, daher keine Repository-Zugangsdaten nötig.

Der bestätigte Produktionsnachweis aus T12 bezieht sich auf Release v0.9, Commit 4b59e03022a3c669136ebd4499b096fcad3efcdb. Die vollständig qualifizierten Image-Digests stehen in docs/validation/t12.json. Start-, Migrations- und Testbefehle stehen im README, Auslieferung und Secrets-Namen in docs/ci-cd.md.

[Prüfzugang offen: Der Verfasser legt fest, ob der Examinator selbst registriert oder ein bestätigtes Testkonto über einen separaten geschützten Kanal erhält. Keine Passwörter oder API-Schlüssel in dieses Dokument, Repository oder ZIP aufnehmen.]

5.4 Betrieb während der Korrekturphase

Die Anwendung muss ab tatsächlichem Abgabetermin mindestens 28 Tage erreichbar bleiben. Der Termin und die verbindliche Betriebsverantwortung sind noch zu bestätigen; deshalb wird die Frist nicht als erfüllt ausgewiesen. docs/submission/README.md enthält das Übergabeblatt, docs/operations.md die Kontrollen und Ausfallbehandlung.

Der Betreiber prüft HTTPS-Bereitschaft, Release-Status, Zertifikat und Backups. Bei Ausfall werden Beginn, Ursache, Massnahme und Wiederverfügbarkeit protokolliert. Wiederanlauf erfolgt über die vorhandenen Compose-/Ansible-Wege ohne Löschen persistenter Volumes. Ein Image-Rollback ist keine Datenbankwiederherstellung.

6 Tests

Die Gesamtabnahme verbindet fachliche Unit-Tests, PostgreSQL-Integration, Browserabläufe, Sicherheitsprüfungen und isolierte Deployment-Tests. Ergebnisse beziehen sich auf den angegebenen Quellstand; nicht ausgeführte Prüfungen sind ausdrücklich getrennt.

6.1 Testvorgehen

T12 prüfte Commit 4b59e03022a3c669136ebd4499b096fcad3efcdb am 22. September 2026: 710 Python-Unit-, 326 PostgreSQL-Integrations-, 65 Chromium-E2E- und 62 JavaScript-Tests bestanden, zusammen 1'163. Architektur, Linting, frische Migration, Build, HTTPS/API-Smoke und die Security-Stufe bestanden innerhalb der dokumentierten Ausnahmen (RepairHub, 2026b).

Der aktive ZAP-Scan lief nur gegen eine isolierte Testinstanz. Ansible-Erstlauf, Wiederholung ohne Änderungen, Konfigurationsupdate, kontrollierter Fehler mit kompatibelem App-Rollback und anschliessende Wiederholung wurden geprüft. Eine Neuerstellung des isolierten Datenbankcontainers erhielt vorhandene Daten.

6.2 Testprotokoll mit 6-12 Testfällen

Die zwölf ausgewählten Fälle fassen die zentralen Anforderungen zusammen. Das ausführliche Protokoll mit Voraussetzungen, Schritten und Nachweisdateien befindet sich unter docs/acceptance-test-protocol.md. Die Tabelle enthält keine zusätzliche Behauptung einer manuellen Durchführung.

Fall	Schwerpunkt	Soll	Ist
A0 1	Registrierung / Verifikation / Reset	Nur bestätigte gültige Identität erhält Zugang.	Bestanden; SMTP- und negative Identitätsfälle geprüft.
A0 2	Anmeldung / Sitzung / Abmeldung	Ungültige Zugänge gesperrt, Abmeldung beendet Zugriff.	Bestanden; CSRF und manipulierte Sitzungen geprüft.
A0 3	Geräte / Eigentum / Listen	Eigene Daten dauerhaft; kein Fremdzugriff; begrenztes DOM.	Bestanden; zwei Konten und virtuelle Fenster geprüft.
A0 4	Reparatur / Schritte / Status	Gültige Änderungen und Wiederaufnahme, keine leeren Texte.	Bestanden; Positiv- und Negativpfade geprüft.
A0 5	Teile / Kosten	CHF 205.00, 160.00 und 0.30; ungültige Zahlen abweisen.	Bestanden; zentrale Decimal-Regel und Grenzen geprüft.
A0 6	Persönliche / System-API	Keys ermöglichen genau den vereinbarten Leseumfang.	Bestanden; Ersetzen/Widerruf und Systemzugang geprüft.
A0 7	API-Fehler	401/404/405 ohne Datenpreisgabe oder Änderung.	Bestanden; Eigentums- und Methodengrenzen geprüft.
A0 8	Migration / Transaktion / Persistenz	Leere DB migriert; Fehler atomar; Daten bleiben erhalten.	Bestanden; PostgreSQL und Container-Neuerstellung geprüft.
A0 9	Browser / Sprache / HTTPS	Durchgängiger Ablauf, DE/EN, verständliche Fehler.	Bestanden; Chromium, mobil, mit/ohne JS; Produktion TLS.
A1 0	Backup / Restore-Verfahren	Backup und nachvollziehbarer Wiederherstellungsweg.	Dokumentiert; tatsächlicher DB-/Gesamtrestore nicht ausgeführt.
A1 1	CI / Security / Deployment	Prüfungen sperren Fehler; identische Releases idempotent.	Bestanden innerhalb der 22 befristeten gosu-Ausnahmen.
A1 2	Optionale Funktionen	Suche, Übersicht, Bilder und PDF zeigen eigene korrekte Daten.	Bestanden; Bildgrenzen, Eigentum und CHF-PDF geprüft.

Tabelle 11 Ausgewählte Abnahmetests

Eigene Zusammenstellung aus T12 (RepairHub, 2026b).

6.3 Ergebnisse und offene Einschränkungen

Die T12-Gesamtanbahnung ist im vereinbarten Umfang bestanden. 22 einzeln befristete gosu-Befunde im PostgreSQL-Image bleiben ausdrücklich akzeptierte Ausnahmen; keine allgemeine Sicherheitsfreigabe. Garage besitzt eine zusätzliche konservative Prüfung seines Rust-Runtime-Abhängigkeitsgraphen. Fünf ZAP-Informationsgruppen waren nicht blockierend.

Nicht belegt sind ein tatsächlicher Datenbank-/Gesamtrestore, der erst nach Abgabe laufende vierwöchige Betrieb sowie die erneute produktive Mailzustellung. Der produktive API-Nachweis traf eine leere Liste; positive Detaildaten wurden isoliert geprüft. Beim Erstellen eines Falls per AJAX erscheint der Bildbereich erst nach erneutem Öffnen der Detailseite; dieser Bedienungshinweis ist in der Anleitung festgehalten.

7 Bedienungsanleitung

Die Bedienung wurde mit Playwright gegen eine isolierte Instanz des geprüften Produktionsimages nachvollzogen. Die vollständige bebilderte Anleitung liegt unter docs/user-guide/README.md und gehört zum Quellcode-ZIP. Die folgenden Abschnitte geben den Einstieg und die browserunabhängigen API-Aufrufe an.

7.1 Bedienung der Webanwendung

Konto erstellen, Bestätigungsmail öffnen und Adresse bestätigen; danach mit E-Mail oder Benutzername anmelden. Unter Deine Geräte ein Gerät mit Name, Hersteller und Modell speichern. Im Gerät einen neuen Reparaturfall mit Fehlerbeschreibung anlegen. Anschliessend Schritte, Status, Arbeitswerte und Ersatzteilpositionen jeweils über ihre eigenen Speicherbuttons nachführen.

Suchfelder reagieren nach kurzer Eingabepause. Statusfilter wirken sofort; lange Geräte- und Falllisten laden beim Scrollen weitere Fenster. Bilddateien in JPEG/PNG/WebP bis 10 MiB und 20 Megapixel werden privat als Mosaik angezeigt und können aus der Galerie entfernt werden. Der PDF-Link lädt den vollständigen Fall samt CHF-Kostenschätzung herunter. Bei vergessenem Passwort wird ein zeitlich begrenzter Reset-Link per E-Mail angefordert.

docs/user-guide/README.md zeigt für sämtliche Benutzerfunktionen die Eingaben, Aktionen und Ergebnisse. Die Demobilder und Demodaten gehören zu einer wegwerfbaren Testinstanz; Passwortfelder und API-Schlüssel sind in den Aufnahmen verdeckt.

7.2 API Authentifizierung und Endpunkte

Im Frontend unter API-Zugriff einen persönlichen API-Schlüssel generieren und sofort geschützt kopieren. Im API-Client den Header Authorization: Bearer <api-key> setzen. Es werden weder Browsercookies noch Benutzername/Passwort mitgeschickt.

GET <https://lab19.ifalabs.org/api/repairs?limit=20&offset=0> liefert items, total, snapshot und next_offset. Für Folgeseiten next_offset und denselben snapshot verwenden; limit liegt zwischen 1 und 60. GET <https://lab19.ifalabs.org/api/repairs/{id}> liefert einen eigenen vollständigen Fall mit Schritten, Teilen und Kosten. {id} durch eine ID der Listenantwort ersetzen.

401 bedeutet fehlenden/ungültigen Key, 404 unbekannten/fremden Fall, 405 nicht erlaubte Schreibmethode und 400 ungültige Parameter. Geldbeträge sind Dezimalstrings in CHF. Vollständige Beispielerantworten und ausführbare Curl-Aufrufe: docs/api.md.

8 Projektabschluss und Reflexion

Die folgenden Angaben trennen belegte Zielerfüllung von noch erforderlicher persönlicher Würdigung. Testergebnisse ersetzen weder die Reflexion des Verfassers noch den tatsächlichen Betrieb während der Korrekturphase.

8.1 Erreichung der Muss- und Kann-Ziele

M01–M07 sowie K01–K04 sind in T12 funktional abgenommen; zusätzlich wurden Passwort-Recovery, Gerätesuche und persönliche API-Key-Verwaltung umgesetzt. Die Abhängigkeitsumkehr und vertikalen Fachdomänen präzisieren den älteren Schichtenentwurf. Ein realer DB-Restore wurde ausdrücklich aus der Ausführung ausgenommen. Die Betriebsfrist ist noch offen.

8.2 Projektverlauf und kritische Würdigung der Lösung

[Persönliche Würdigung durch den Verfasser ergänzen. Belegbare Themen: Nutzen und Aufwand der Slice-Ports; gemeinsames Deployment eines modularen Monolithen; Einzelhost

als Ausfallpunkt; zusätzliche Garage-/Backup-Komplexität; Grenze zwischen bestandenen Tests und nicht durchgeführtem Restore. Keine erfundenen Arbeitsstunden oder persönlichen Erfahrungen eintragen.]

8.3 Persönliche Reflexion und Ausblick

[Vom Verfasser in eigenen Worten ergänzen: wichtigste Lernerfahrung, konkrete Schwierigkeiten und deren Lösung, Abweichungen vom Zeitplan, verbleibende Risiken und begründete nächste Schritte.]

9 Abkürzungsverzeichnis

API – Application Programming Interface; CI/CD – Continuous Integration/Continuous Delivery; CSRF – Cross-Site Request Forgery; DAST – Dynamic Application Security Testing; DTO – Data Transfer Object; ERD – Entity Relationship Diagram; ORM – Object-Relational Mapping; SAST – Static Application Security Testing; SMTP – Simple Mail Transfer Protocol; TLS – Transport Layer Security; UTC – Coordinated Universal Time.

10 Glossar

Adapter: technische Implementierung eines vom Anwendungsfall definierten Ports. Port: frameworkfreier Vertrag für benötigte externe Fähigkeiten. Vertikaler Slice: vollständiger fachlicher Anwendungsfall mit eigenen Eingaben, Ablauf und Schnittstellen. Humble Object: kleine technische DOM-Anbindung; Verhalten bleibt unabhängig testbar. Image-Digest: unveränderliche Inhaltskennung eines Containerimages. Idempotenz: erneuter Abgleich gleicher Eingaben erzeugt keine unnötigen Änderungen.

11 Abbildungsverzeichnis

Abbildung 1 Use-Case-Übersicht zu Rollen und Bereichen.....	11
Abbildung 2 Use-Case-Details zu Reparaturbearbeitung und Kosten.....	12
Abbildung 3 Verantwortlichkeiten und Abhängigkeitsumkehr.....	14
Abbildung 4 Komponenten und Abhängigkeiten von RepairHub.....	15
Abbildung 5 Relationales Datenmodell von RepairHub.....	17
Abbildung 6 Aktivitätsdiagramm zur Reparaturbearbeitung.....	19
Abbildung 7 Bereitstellung von RepairHub mit Docker Compose.....	20
Abbildung 8 Persönlichen API-Schlüssel erzeugen.....	22
Abbildung 9 API-Sequenz zum geschützten Datenabruf.....	23

12 Tabellenverzeichnis

Tabelle 1 Muss-Ziele von RepairHub.....	3
Tabelle 2 Kann-Ziele von RepairHub.....	3
Tabelle 3 Aufwandsplanung.....	4
Tabelle 4 Projektrisiken und Gegenmassnahmen.....	5
Tabelle 5 Funktionale Anforderungen und Szenarien.....	8
Tabelle 6 Nichtfunktionale Anforderungen und Szenarien.....	10
Tabelle 7 Anwendungsfälle und Abnahmekriterien.....	13
Tabelle 8 Technologieauswahl und Begründung.....	16
Tabelle 9 Ansichten der Benutzeroberfläche.....	18
Tabelle 10 Sicherheitsmassnahmen für RepairHub.....	24
Tabelle 11 Ausgewählte Abnahmetests.....	26

13 Quellenverzeichnis

RepairHub (2026a). Implementierter Quellstand und technische Dokumentation. Eigener Projektbestand, Commit 4b59e03022a3c669136ebd4499b096fcad3efcdb. <https://github.com/GojicDragan/RepairHub>; insbesondere docs/domain-architecture.md, docs/api.md, docs/ci-cd.md, docs/images.md und docs/pdf-reports.md.

RepairHub (2026b). Gesamtabnahme und Testprotokoll vom 22. September 2026. Eigene Prüfergebnisse: docs/acceptance-test-protocol.md, docs/t12-validation.md und docs/validation/t12.json.

RepairHub (2026c). Bebilderte Benutzeranleitung. Eigene Playwright-Aufnahmen der isolierten Anwendung: docs/user-guide/README.md; Ablauf unter scripts/documentation/capture_user_guide.py. Stand: 22. September 2026.

Docker Inc. (o. J. a). Networking in Compose. Abgerufen am 7. September 2026 von <https://docs.docker.com/compose/how-tos/networking/>

Docker Inc. (o. J. b). Use Compose in production. Abgerufen am 7. September 2026 von <https://docs.docker.com/compose/how-tos/production/>

Docker Inc. (o. J. c). Volumes. Abgerufen am 7. September 2026 von <https://docs.docker.com/engine/storage/volumes/>

F5 (o. J.). Module ngx_http_proxy_module. Abgerufen am 7. September 2026 von https://nginx.org/en/docs/http/ngx_http_proxy_module.html

Flask-Login (o. J.). Flask-Login Documentation. Abgerufen am 7. September 2026 von <https://flask-login.readthedocs.io/en/latest/>

Flask-WTF (o. J.). Flask-WTF Documentation. Abgerufen am 7. September 2026 von <https://flask-wtf.readthedocs.io/en/1.2.x/>

Grinberg, M. (o. J.). Flask-Migrate Documentation. Abgerufen am 7. September 2026 von <https://flask-migrate.readthedocs.io/en/latest/>

Pallets (o. J.). Flask Documentation. Abgerufen am 7. September 2026 von <https://flask.palletsprojects.com/en/stable/>

PostgreSQL Global Development Group (o. J.). Numeric Types. Abgerufen am 7. September 2026 von <https://www.postgresql.org/docs/current/datatype-numeric.html>

SQLAlchemy (o. J.). SQLAlchemy ORM. Abgerufen am 7. September 2026 von <https://docs.sqlalchemy.org/en/20/orm/>

Anhänge

Anhang A: docs/user-guide/README.md mit Bildschirmaufnahmen und PDF-Beispiel.
Anhang B: docs/acceptance-test-protocol.md und docs/validation/t12.json. Anhang C: docs/diagrams mit bearbeitbaren PlantUML-Quellen. Anhang D: docs/submission/README.md als Übergabebblatt; Zugangsdaten separat übergeben.

Eigenständigkeitserklärung und Deklaration des KI-Einsatzes

Vorlage gemäss ipso!-Leitfaden, Abschnitt 1.4.13. Erst nach eigener Prüfung durch den Verfasser ausfüllen und unterschreiben.

Ich bestätige, dass ich diese Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen dafür benutzt habe. Alle Stellen, die wörtlich oder sinngemäss übernommen oder mittels generativen KI-Systemen erstellt wurden, habe ich als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form weder veröffentlicht noch einer anderen Prüfungsbehörde vorgelegt.

Ort, Datum: _____ Vorname Name: _____

Unterschrift: _____

Entwurf der KI-Deklaration; tatsächlichen Einsatz vor Unterzeichnung prüfen und vervollständigen:

KI-Werkzeug	Einsatzzweck	Einsatzbereich
Codex	Programmierung, Kommentare und Testunterstützung	Quellcode / technische Kapitel 4–6
Codex	Abgleich vorhandener Unterlagen mit Implementierung und Prüfergebnissen	Kapitel 2–7, Zielerfüllung 8.1, Verzeichnisse
Codex mit Playwright	Ausführung der Demoabläufe und Bildschirmaufnahmen	Benutzeranleitung / Anhang A